

# Interview Task: Archive File Extractor Service

## Overview

Build an HTTP service, packaged as a Docker container, that accepts archive files and a file pattern, searches for matching files — including inside nested archives — and stores the results in a database table.

**Estimated time:** 2 hours **Submission:** Git repository with source code, Dockerfile, README, and tests.

---

## The Task

Implement a service that exposes an HTTP API. Given a zip archive and a file pattern, the service finds every file inside the archive whose path matches the pattern, persists the results to a database, and exposes the results via API.

## Configuration

The service is configured via environment variables (or a config file):

- `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_TABLE`
- `CONCURRENCY` — number of parallel workers (optional, with a sensible default)
- `PORT` — HTTP port to listen on

## Suggested API

**POST /extractions** Submit a new extraction job.

- Accepts the archive as `multipart/form-data` upload **or** a URL/path reference — your choice, justify in the README
- Body parameters: `pattern` (glob, required)
- Response: `202 Accepted` with a `job_id`

**GET /extractions/{job\_id}** Get job status (`pending` / `running` / `completed` / `failed`) and summary (number of matches, error if any).

**GET /extractions/{job\_id}/results** List matched files for a job, with pagination.

**GET /health** Liveness/readiness endpoint.

You may adjust the API shape — what matters is that the service is usable, jobs can be tracked, and results retrieved.

## Example usage

Shell

```
# Start the service
docker run --rm -p 8080:8080 \
  -e DB_HOST=db.example.com -e DB_PORT=5432 \
  -e DB_NAME=extractor -e DB_USER=user -e DB_PASSWORD=*** \
  -e DB_TABLE=extracted_files \
  file-extractor-service

# Submit a job
curl -X POST http://localhost:8080/extractions \
  -F "archive=@./input.zip" \
  -F "pattern=**/*.json"

# Check status
curl http://localhost:8080/extractions/<job_id>

# Get results
curl http://localhost:8080/extractions/<job_id>/results
```

---

## Requirements

### 1. Nested archives

Archives may contain other archives (zip, tar, tar.gz, etc.) at **any nesting level**. The service must recursively descend into all of them and continue matching the pattern. Assume nesting can be arbitrarily deep; enforce a safety limit of your choice.

### 2. Concurrency

The service should leverage concurrency to reduce processing time — for example, processing multiple nested archives in parallel, or scanning entries within an archive in parallel. It should also be able to handle multiple extraction jobs concurrently. Be mindful of:

- Race conditions on shared state
- Connection pooling / safe DB writes
- Cleanup of temporary files
- Bounded resource usage (don't spawn unlimited goroutines/threads)

### 3. Asynchronous job processing

Submitting a job should not block the HTTP request until extraction finishes. The job runs in the background and progress is queryable via the status endpoint. The choice of in-process worker pool, queue, or another mechanism is yours — justify it in the README.

### 4. Database storage

For every matching file, insert a row into the target table. You may design the schema, but it must capture at minimum:

- Job ID (link to the extraction job)
- The full path to the file, including the nesting chain (e.g. `outer.zip/inner.tar.gz/data/file.json`)
- File name and size
- Nesting depth
- Timestamp of extraction
- Source archive name

You should also store the jobs themselves (id, status, submitted\_at, completed\_at, error, etc.). Include a setup script (DDL / migration) for creating the tables.

### 5. Containerization

The service must run as a Docker container. The image should be minimal and require no external setup beyond `docker run` (plus a reachable database). A `docker-compose.yml` that brings up the service together with a database is a nice touch.

---

## Non-Functional Expectations

- Clear error handling: corrupt archives, unsupported formats, DB failures, invalid input — informative responses, appropriate HTTP status codes, no resource leaks
- Structured logging with job IDs for traceability

- Reasonable memory usage on large archives (prefer streaming over loading into memory)
  - Temporary files cleaned up, including on failure
  - Graceful shutdown: in-flight jobs should be handled sensibly when the service is stopped
- 

## Deliverables

1. Python
  2. `Dockerfile`
  3. Flask
  4. `README.md` with build & run instructions, API documentation, examples, and any assumptions you made
  5. Tests — unit tests, plus at least one integration test exercising the full flow against a sample archive with nested archives
- 

## Submission

Please send us a link to a Git repository (GitHub/GitLab/Bitbucket) containing your solution. In your README, include:

- How to build and run the service
- How to exercise it against a sample archive (feel free to include one in the repo)
- Any assumptions or shortcuts you made and what you would do differently with more time

Good luck - we're looking forward to seeing your solution and discussing it with you.